

SubMagic Avatar POC

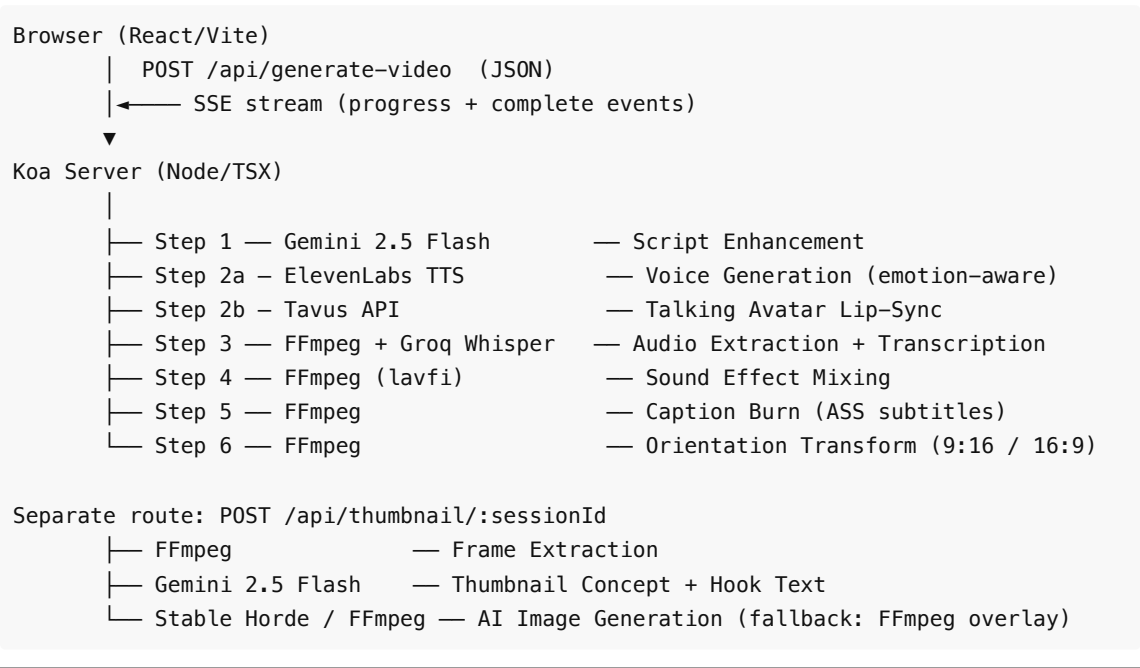
Overview

A proof-of-concept pipeline that turns a text script into a fully produced short-form video featuring a talking avatar — with AI-enhanced narration, lip-synced video, word-level captions, optional sound effects, and an AI-generated thumbnail.

Stack: Vite + React 19 (frontend) · Koa (backend) · TypeScript end-to-end

Delivery: Server-Sent Events (SSE) for real-time progress streaming

Architecture



ElevenLabs — Voice

What it does in this POC

ElevenLabs converts the AI-enhanced script into a natural-sounding MP3 that drives the avatar's lip-sync. Unlike the previous macOS `say` approach, ElevenLabs voices are human-quality and respond to emotion-tuned parameters.

Free Plan

Feature	Free Plan
Characters / month	10,000
Voices available	ElevenLabs built-in "premade" voices
Voice Library (community voices)	API blocked — requires paid plan

Models	eleven_multilingual_v2, eleven_turbo_v2_5
Voice cloning	Not available

Important: Voice Library voices (Adam, Rachel, Bella, Josh, etc.) are blocked via API on the free plan. This POC dynamically fetches the account's available voices at runtime (`GET /v1/voices`), filters to premade voices, and picks by name preference — so it works on free and paid plans without code changes.

Voice Mapping

UI Voice Option	Preferred ElevenLabs Voice	Character
Professional Male	Brian	Deep, authoritative American male
Professional Female	Alice	Clear, polished British female
Friendly Female	Sarah	Warm, soft American female
Motivational Male	Eric	Confident, energetic American male

Preferences are tried in order. If a preferred name is not in the account, the next candidate is tried, and finally the first available voice is used as a hard fallback.

Emotion → Voice Settings

ElevenLabs exposes three parameters that control delivery style:

- **stability** — `0` = very expressive and variable · `1` = flat and robotic
- **style** — style exaggeration multiplier (0–1); amplifies the voice's inherent character
- **similarity_boost** — how closely the output adheres to the original voice
- **use_speaker_boost** — improves clarity and presence (disabled for soft emotions)

Emotion	Stability	Style	Similarity	Speaker Boost
Neutral	0.71	0.00	0.50	Yes
Happy	0.50	0.30	0.75	Yes
Excited	0.30	0.55	0.75	Yes
Motivational	0.38	0.48	0.75	Yes
Professional	0.82	0.00	0.80	Yes
Sad	0.65	0.20	0.65	No

Model used: **eleven_multilingual_v2** — highest quality on the free plan, supports all languages.

How the audio reaches Tavus

Tavus can accept either a text script (and does its own TTS) or a public `audio_url` for lip-sync. When ElevenLabs audio is generated:

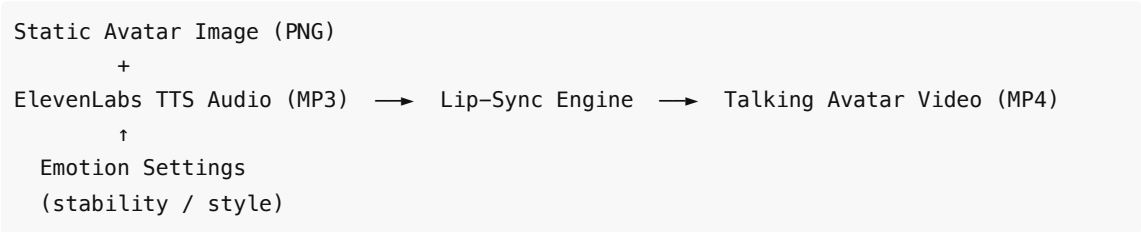
1. The MP3 is saved to `tmp/{sessionId}/tts.mp3`
2. The Koa server exposes it at `GET /api/audio/{sessionId}`

- 3. If `SERVER_BASE_URL` is configured, `audio_url = SERVER_BASE_URL/api/audio/{sessionId}` is passed to Tavus
- 4. Tavus lip-syncs the avatar to the ElevenLabs audio instead of generating its own voice
- 5. If `SERVER_BASE_URL` is not set (local dev without a tunnel), Tavus falls back to its own TTS

ElevenLabs — Image to Video

Concept

"Image to Video" refers to the workflow of taking a **static avatar image** and transforming it into a **talking head video** using an audio track. ElevenLabs is the voice layer in this pipeline; the image animation is handled by a lip-sync service.



Why ElevenLabs enables better image-to-video quality

Without ElevenLabs	With ElevenLabs
Generic TTS (e.g. macOS say) — flat, robotic	Natural, human-quality voice
No emotion control	Stability + style settings per emotion
Fixed voice, no options	4 distinct voice personas (male/female, professional/friendly)
Lip-sync driven by mediocre audio	Cleaner phoneme boundaries → sharper lip movement

Full Pipeline: Static Image → Talking Video

Step	Service	Input	Output
1	Gemini 2.5 Flash	Raw script + emotion	Polished script
2a	ElevenLabs	Script + voice + emotion	MP3 audio
2b	Tavus	Avatar image + MP3 URL	Talking avatar MP4
3	Groq Whisper	Avatar MP4 audio	Word timestamps
4	FFmpeg	Avatar MP4 + timestamps	SFX-mixed MP4
5	FFmpeg	SFX MP4 + ASS subtitles	Captioned MP4
6	FFmpeg	Captioned MP4	Final portrait/landscape MP4

Alternative Lip-Sync Backends (future)

The ElevenLabs audio output (an MP3 buffer) is backend-agnostic. If Tavus is replaced, the same audio feeds directly into:

Service	Type	Notes
fal.ai OmniHuman	Cloud API	High realism, accepts audio URL
SadTalker	Local model	Python, GPU required
LivePortrait	Local model	Faster than SadTalker, portrait-optimised
Wav2Lip	Local model	Older, widely supported

Pipeline Steps (Full)

Step 1 — Script Enhancement (Gemini 2.5 Flash)

The user's raw script is sent to Gemini with emotion and duration target (15/30/60/90s → 37/75/150/225 words). Gemini rewrites it for the right length and emotional tone. Falls back to the original script if Gemini is unavailable.

Token usage is captured from `response.usageMetadata` and returned alongside the text.

Step 2a — Voice Generation (ElevenLabs)

Enhanced script → ElevenLabs TTS → MP3 saved to session dir. Voice and emotion settings as described above. If ElevenLabs fails, the pipeline continues without audio URL (Tavus uses its own TTS as fallback).

Step 2b — Talking Avatar (Tavus)

Submits to Tavus `/v2/videos` with either `audio_url` (ElevenLabs) or `script` (fallback). Polls every 6 seconds until `status = "ready"` (10 min timeout), then downloads the MP4.

Step 3 — Transcription (Groq Whisper)

Audio extracted from the avatar MP4 as WAV. Groq `whisper-large-v3` transcribes with word-level timestamps used for caption positioning and SFX alignment.

Step 4 — Sound Effects (FFmpeg lavfi)

Three synthesised effects mixed at user-selected word timestamps:

Effect	Generator	Duration
Pop	Damped sine 1 kHz	120ms
Ding	Damped sine 1.5 kHz	300ms
Whoosh	Bandpass white noise	250ms

Step 5 — Caption Burn (FFmpeg + ASS)

Word timestamps → `.ass` subtitle file → burned into video via `libx264` re-encode. Three style presets (Viral, Professional, Creator), fully customisable per session.

Step 6 — Orientation Transform (FFmpeg)

- **Landscape (16:9):** stream copy, no re-encode
- **Portrait (9:16):** scale to 1080px width, pad to 1080×1920 with black bars

Thumbnail Generation (Separate Route)

1. **Frame extraction** — clean frame from `avatar.mp4` at a style-specific timestamp
2. **Gemini Vision concept** — multimodal prompt with frame + script → `hookLine` , `subLine` , Stable Diffusion prompt
3. **Image generation:**
 - **Primary:** Stable Horde (free, anonymous, FLUX/SDXL, 512×320, 30s timeout)
 - **Fallback:** FFmpeg `drawtext` / `drawbox` overlay on extracted frame

Five styles: Engaging, Normal, Cinematic, Professional, Dynamic.

AI Token Usage

Token usage is tracked per Gemini call and displayed in the UI after generation.

Call	Location	Tracked
enhanceScript	gemini.ts	Returns { text, tokens }
generateConcept (thumbnail)	thumbnail.ts	Logged to console

Token counts from `response.usageMetadata` : `promptTokenCount` (input), `candidatesTokenCount` (output), `totalTokenCount` .

Typical per-video usage

Step	Input tokens	Output tokens
Script Enhancement	300–600	75–250
Thumbnail Concept	800–1,500	200–400

ElevenLabs, Groq Whisper, and Tavus are not token-based — they bill by characters, audio duration, and video seconds respectively.

Services & APIs

Service	Used for	Plan	Auth
ElevenLabs	TTS with emotion settings	Free (10k chars/mo)	ELEVENLABS_API_KEY
Gemini 2.5 Flash	Script enhancement, thumbnail concept	Free tier	GEMINI_API_KEY
Tavus	Talking avatar lip-sync	Paid	TAVUS_API_KEY

Groq Whisper	Audio transcription + word timestamps	Free tier	GROQ_API_KEY
Stable Horde	Free AI thumbnail generation	Free, anonymous	None
FFmpeg	Audio extraction, SFX, captions, orientation	Local binary	None

Environment Variables

```
ELEVENLABS_API_KEY=      # ElevenLabs – free plan works
GEMINI_API_KEY=          # Google AI Studio – free
GROQ_API_KEY=            # Groq – free
TAVUS_API_KEY=           # Tavus
TAVUS_REPLICA_ID=        # fallback replica for all avatars
TAVUS_REPLICA_ID_AVATAR1= # per-avatar overrides (optional)
TAVUS_REPLICA_ID_AVATAR2=
TAVUS_REPLICA_ID_AVATAR3=
TAVUS_REPLICA_ID_AVATAR4=

# Required for ElevenLabs audio → Tavus lip-sync
# Tavus must fetch the MP3 from a public URL.
# Local dev: npx localtunnel --port 3001 → set SERVER_BASE_URL=https://xxx.loca.lt
SERVER_BASE_URL=
```

Running Locally

```
npm install
npm run dev      # Vite on :5173 + Koa on :3001

# To enable ElevenLabs → Tavus lip-sync locally:
npx localtunnel --port 3001
# Add SERVER_BASE_URL=https://xxx.loca.lt to .env
```

Known Limitations / POC Scope

- **SERVER_BASE_URL** required for full ElevenLabs integration — without it, Tavus cannot fetch the audio and falls back to its own TTS. Any ngrok/localtunnel/public server URL works.
- **Free ElevenLabs plan = 10,000 chars/month** — a typical 30s script is ~300–400 chars; that's ~25 videos/month on the free tier.
- **Tavus latency** — avatar rendering takes 60–180s. No queue management or webhooks; pure polling.
- **Single-user sessions** — no auth, no persistence. Session files live in `tmp/` and deleted on "Generate Another".
- **Stable Horde availability** — anonymous free tier has variable queue times. FFmpeg fallback is always used if Horde times out (30s).

- **analyzeSoundEffects** — Gemini-based auto sound effect placement is implemented but not wired into the main pipeline. Sound effects are user-selected only.